

Busca em largura

Inicialização

$t \leftarrow 0$ -- t é o relógio ou tempo global

$F \leftarrow \emptyset$ -- fila auxiliar para a busca em largura

para todo vértice v em $V(G)$ faça

$L(v) \leftarrow 0$ -- $L(v)$ é o índice de v na busca em largura

$\text{pai}(v) \leftarrow \text{null}$ -- ponteiros que definem a floresta de largura

Para que a busca atinja todos os vértices, no caso de G ser desconexo:

enquanto existe v em $V(G)$ tal que $L(v) = 0$ faça

$\text{nivel}(v) \leftarrow 0$ -- como v é uma nova raiz, seu nível é igual a 0

$t \leftarrow t + 1$; $L(v) \leftarrow t$

 colocar v na fila F

 realizar a busca em largura

fim-enquanto

Busca em largura

Algoritmo iterativo para a busca em largura:

enquanto $F \neq \emptyset$ faça

$v \leftarrow$ primeiro elemento de F

 retirar v de F

para todo vértice w em $N(v)$ faça

se $L(w) = 0$

então visitar aresta vw

 --aresta “pai” da floresta de largura T

$\text{pai}(w) \leftarrow v$

 -- v é o pai de w na floresta de largura T

$\text{nível}(w) \leftarrow \text{nível}(v) + 1$; $t \leftarrow t + 1$; $L(w) \leftarrow t$

 colocar w no final da fila F

senão se $\text{nível}(w) = \text{nível}(v)$

então se $\text{pai}(w) = \text{pai}(v)$

então vw é aresta “irmão”

senão vw é aresta “primo”

} visitar a aresta
somente se w
ainda está em F

senão se $\text{nível}(w) = \text{nível}(v) + 1$

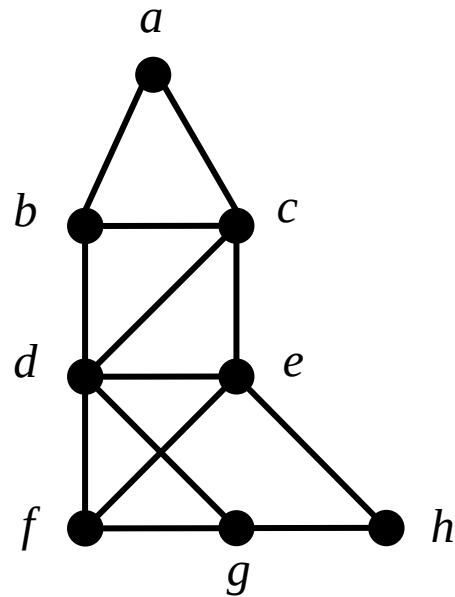
então vw é aresta “tio”

fim-para

fim-enquanto

Busca em largura

G

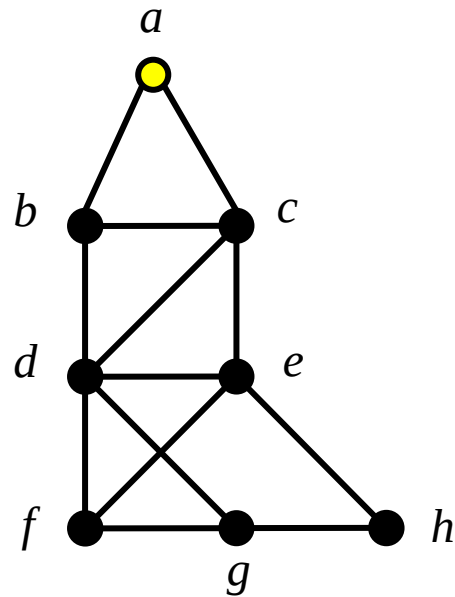


***F* = ∅**

<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>L(v)</i>	0	0	0	0	0	0	0	0

Busca em largura

G

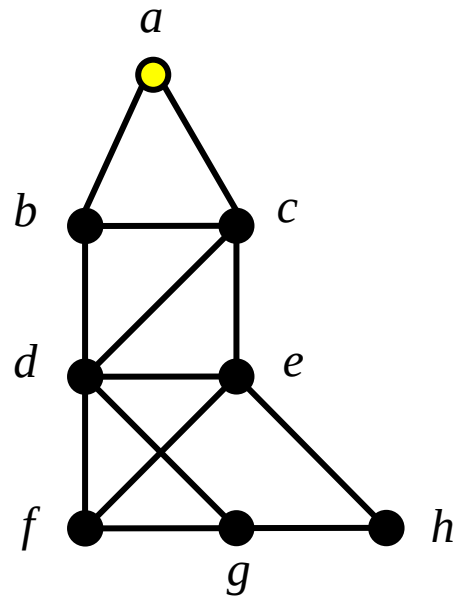


$$F = a$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	0	0	0	0	0	0	0

Busca em largura

G

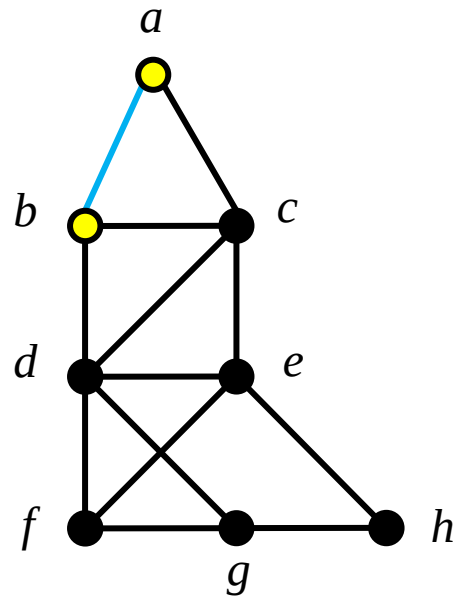


$$F = \emptyset$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	0	0	0	0	0	0	0

Busca em largura

G

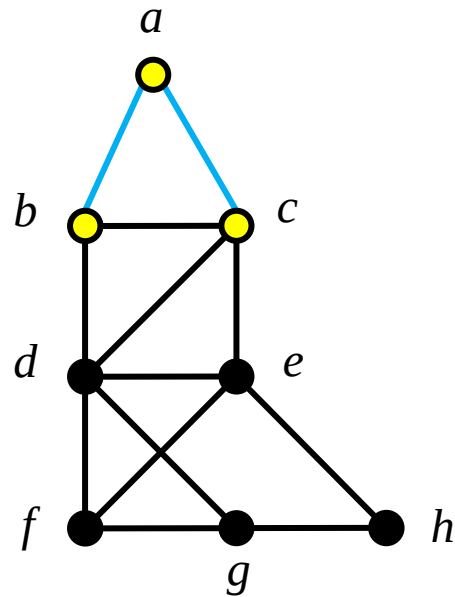


$$F = \cancel{a} b$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	0	0	0	0	0	0

Busca em largura

G

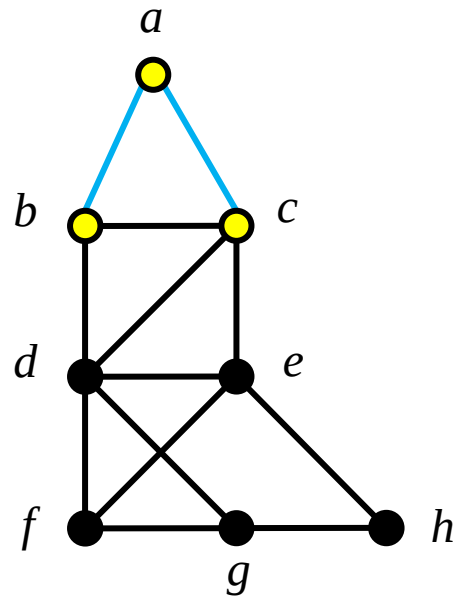


$$F = \cancel{a} b c$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	0	0	0	0	0

Busca em largura

G

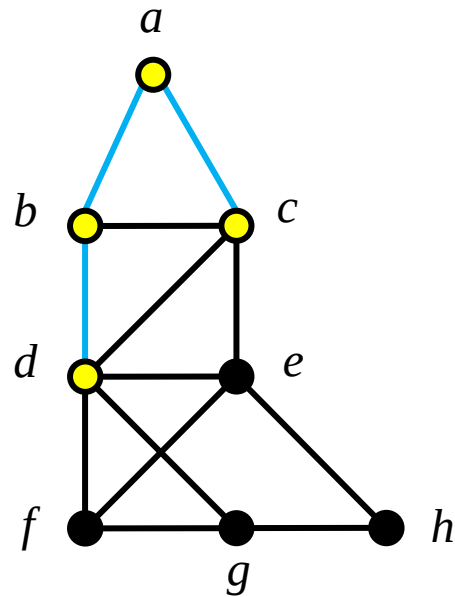


$$F = \cancel{a} \cancel{b} c$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	0	0	0	0	0

Busca em largura

G

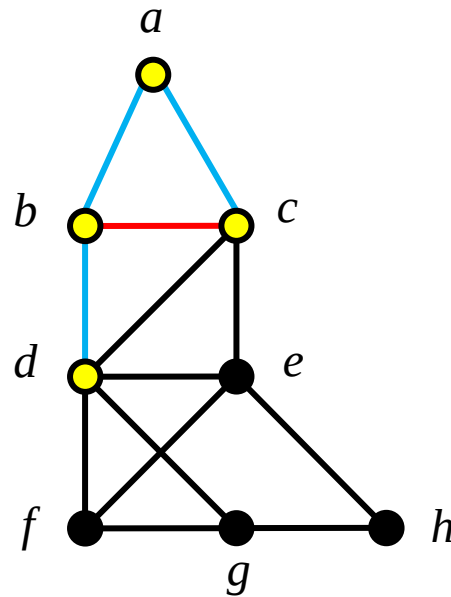


$$F = \cancel{a} \cancel{b} c d$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	0	0	0	0

Busca em largura

G

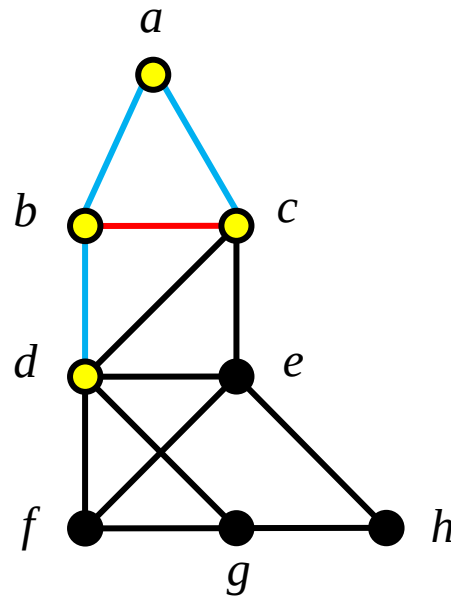


$F = \cancel{a} \cancel{b} c d$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	0	0	0	0

Busca em largura

G

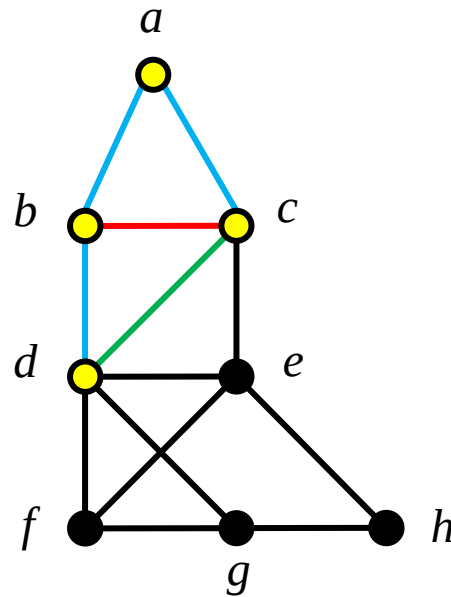


$$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d}$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	0	0	0	0

Busca em largura

G

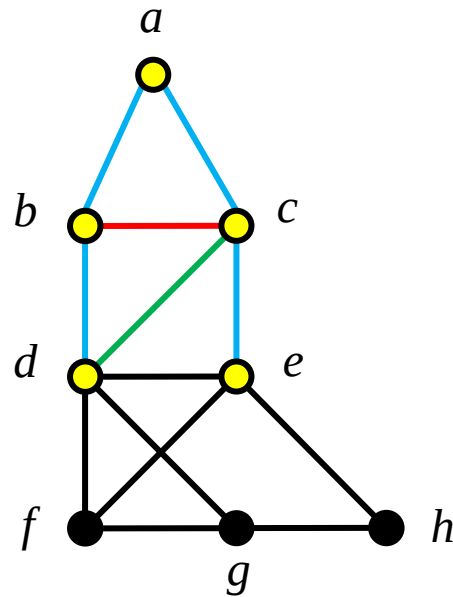


$$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d}$$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	0	0	0	0

Busca em largura

G

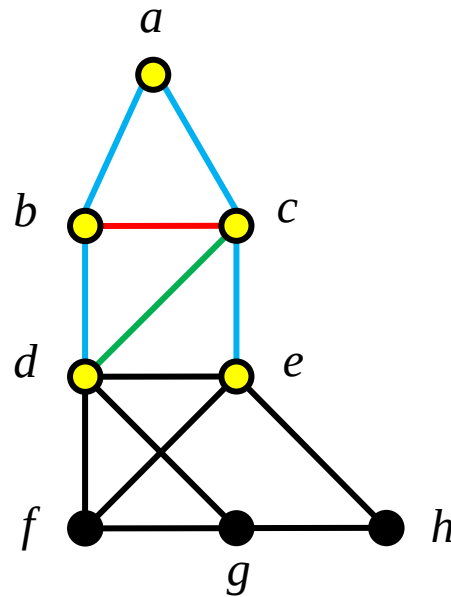


$F = \cancel{a} \cancel{b} \cancel{c} d e$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	0	0	0

Busca em largura

G

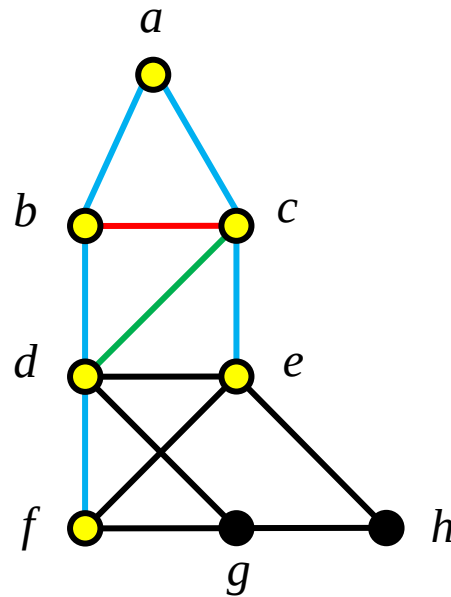


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} \cancel{e}$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	0	0	0

Busca em largura

G

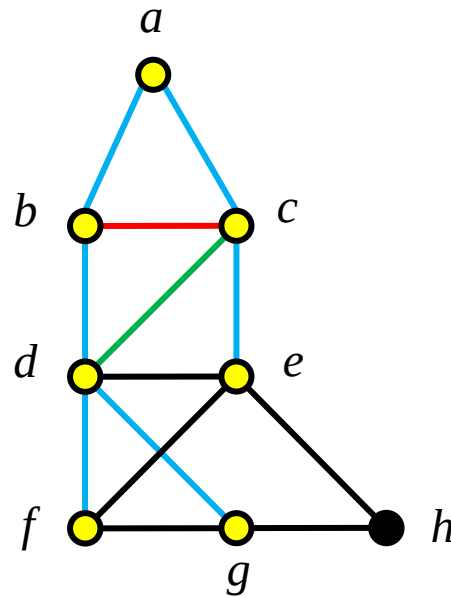


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} e f$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	0	0

Busca em largura

G

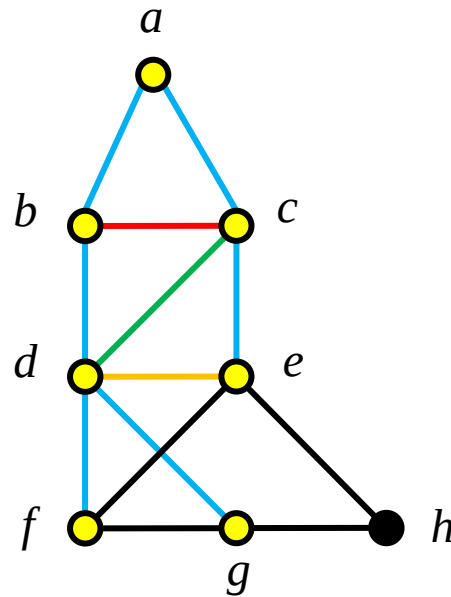


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} e f g$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	0

Busca em largura

G

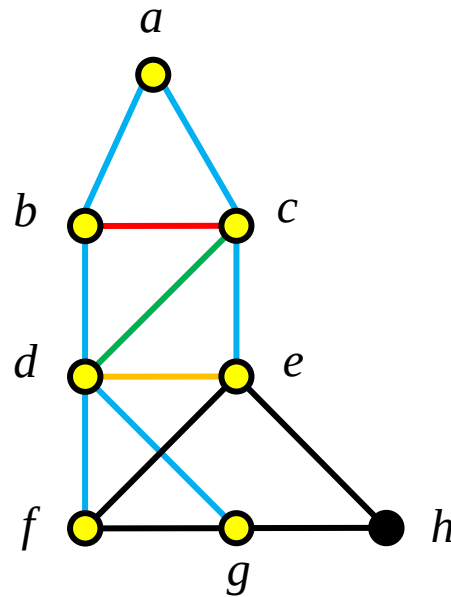


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} e f g$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	0

Busca em largura

G

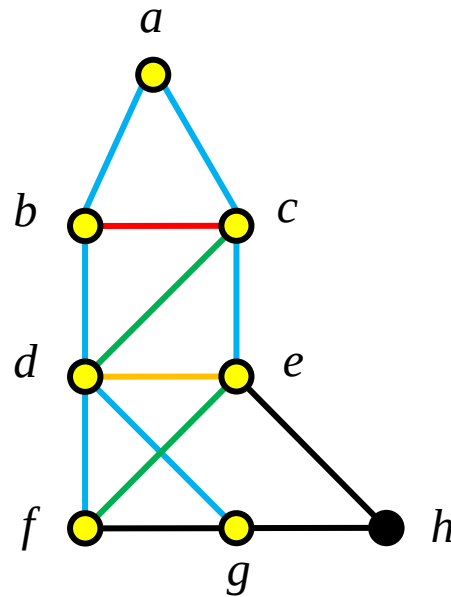


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} \cancel{e} f g$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	0

Busca em largura

G

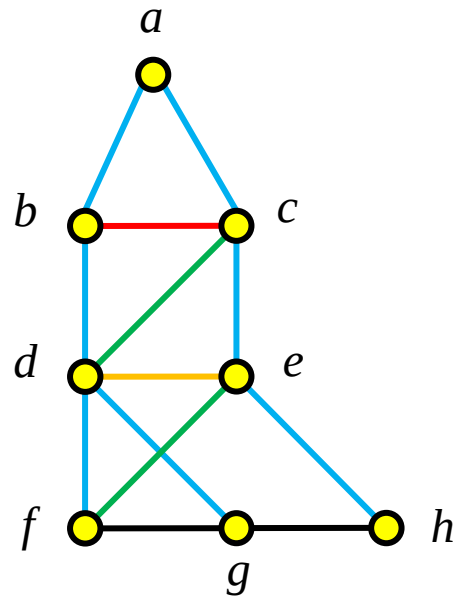


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} \cancel{e} f g$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	0

Busca em largura

G

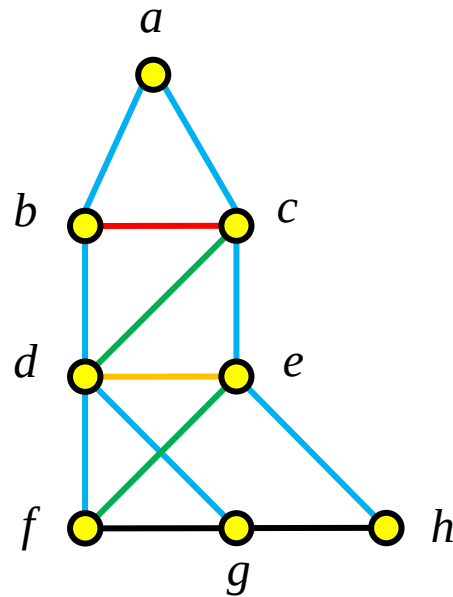


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} \cancel{e} f g h$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	8

Busca em largura

G

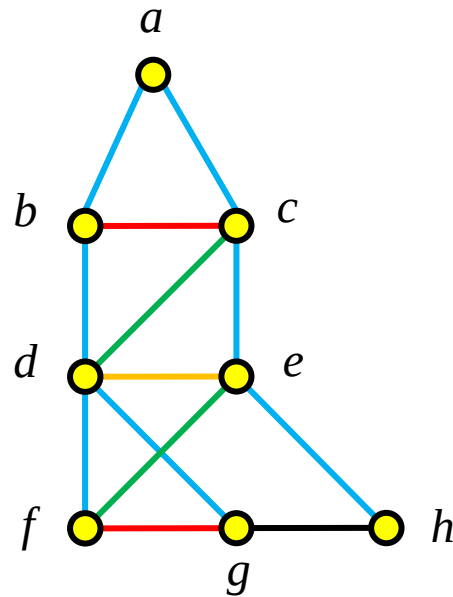


F = ~~a~~ ~~b~~ ~~c~~ ~~d~~ ~~e~~ ~~f~~ ~~g~~ ~~h~~

<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>L(v)</i>	1	2	3	4	5	6	7	8

Busca em largura

G

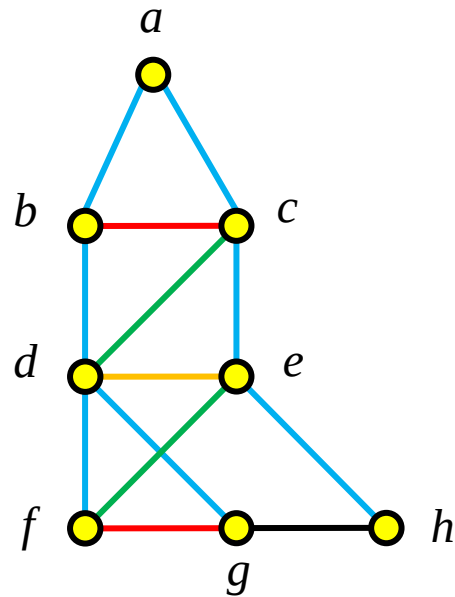


F = ~~*a*~~ ~~*b*~~ ~~*c*~~ ~~*d*~~ ~~*e*~~ ~~*f*~~ *g* *h*

<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>L(v)</i>	1	2	3	4	5	6	7	8

Busca em largura

G

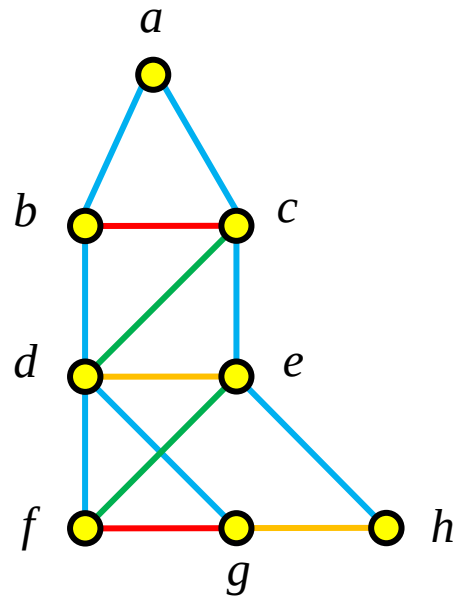


$F = \cancel{a} \cancel{b} \cancel{c} \cancel{d} \cancel{e} \cancel{f} \cancel{g} h$

v	a	b	c	d	e	f	g	h
$L(v)$	1	2	3	4	5	6	7	8

Busca em largura

G

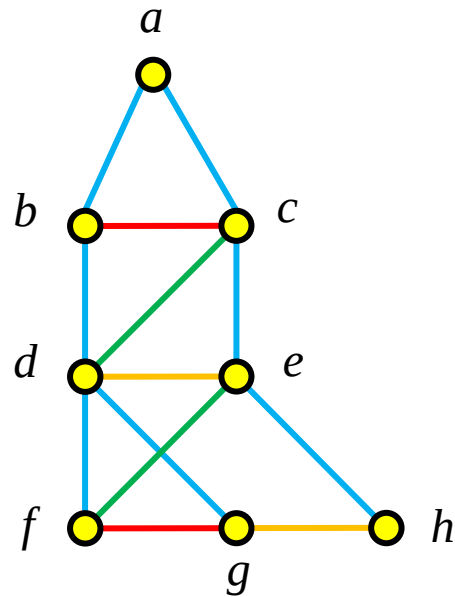


F = ~~*a*~~ ~~*b*~~ ~~*c*~~ ~~*d*~~ ~~*e*~~ ~~*f*~~ ~~*g*~~ *h*

<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>L(v)</i>	1	2	3	4	5	6	7	8

Busca em largura

G



F = ~~*a*~~ ~~*b*~~ ~~*c*~~ ~~*d*~~ ~~*e*~~ ~~*f*~~ ~~*g*~~ ~~*h*~~

<i>v</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>h</i>
<i>L(v)</i>	1	2	3	4	5	6	7	8

Busca em largura

“Esgotar o pai antes de processar os filhos”

“A próxima aresta a ser visitada parte sempre do vértice **mais antigo** na busca”

Busca em largura

Complexidade da busca em largura:

$$O(n + m)$$

(onde $n = |V(G)|$ e $m = |E(G)|$)

Busca em largura

- A floresta de largura T é uma **floresta geradora de G** (isto é, uma floresta que alcança todos os vértices de G); neste caso, todos os vértices de G são alcançados pela busca e ficam com um valor $L(v)$ diferente de zero no final da mesma.
- Somente as arestas-*pai* (**azuis**) (ligando pai e filho) pertencem à floresta de profundidade T . As arestas-*irmão* (**vermelhas**), *primo* (**amarelas**) e *tio* (**verdes**) não pertencem a T .

Busca em largura

Caracterização das arestas-pai (arestas azuis)

Seja vw aresta de G . Então:

vw é uma aresta-pai (da floresta de largura)
se e somente se

$nível(w) = nível(v) + 1$ e, no momento da visita, $L(w) = 0$

Busca em largura

Caracterização das arestas-tio (arestas verdes)

Seja vw aresta de G . Então:

vw é uma aresta-tio

se e somente se

$nível(w) = nível(v) + 1$ e, no momento da visita, $L(w) \neq 0$

Busca em largura

Caracterização das arestas-irmão (arestas vermelhas)

Seja vw aresta de G . Então:

vw é uma aresta-irmão
se e somente se
 $nível(w) = nível(v)$ e $pai(w) = pai(v)$

Busca em largura

Caracterização das arestas-primos (arestas amarelas)

Seja vw aresta de G . Então:

vw é uma aresta-primos
se e somente se
 $nível(w) = nível(v)$ e $pai(w) \neq pai(v)$

Busca em largura

Propriedade fundamental da busca em largura

Seja vw uma aresta de um grafo G , e seja T uma floresta de largura de G . Então:

$$| \text{nível}(v) - \text{nível}(w) | \leq 1.$$

- A demonstração dessa propriedade se reduz ao caso em que vw é uma aresta-tio em relação a T .
- Os vértices que se encontram em F em qualquer momento da busca diferem em no máximo um nível!

Busca em largura

Aplicação 1: Dado um grafo conexo G e um vértice x de G , determinar as *distâncias de x* a todos os demais vértices.

Solução:

- Aplicar uma busca em largura em G com raiz x , obtendo uma árvore de largura T
- Ao final da busca, $dist(x, v) = nível(v)$, para todo v
- Um caminho mínimo de x a v é dado pelo caminho de x a v na árvore T .

Busca em largura

Aplicação 2: Dado um *labirinto* representado por uma matriz, determinar o menor caminho da entrada até a saída do labirinto (se existir).

Solução:

- Modelar a matriz como um grafo, onde a entrada e a saída são representadas por vértices x e y .
- Realizar uma busca em largura em G com uma única raiz x , obtendo uma árvore de largura T .
- Ao final da busca, se $L(y) \neq 0$ então y pode ser alcançado a partir de x .
- O caminho de x a y em T é a solução, e $dist(x, y) = nível(y)$.

Busca em largura

Aplicação 3: Dado um grafo G , determinar se G é bipartido.

Solução:

- Aplicar uma busca em largura em G uma raiz qualquer, obtendo uma floresta de largura T .
- Ao final da busca, G é bipartido se e somente se T não contém **arestas-irmão** nem **arestas-primo**.
- Note que arestas-irmão e arestas-primo fecham **ciclos ímpares!**
- Para definir uma 2-coloração de G : os vértices em níveis pares recebem uma cor, e os vértices em níveis ímpares outra cor.

Busca em largura

Exercício: Elaborar um método (algoritmo) para resolver a seguinte questão: Dado um grafo conexo G e uma árvore geradora T de G , decidir se T é uma árvore de largura para G . Isto é, decidir se existe uma busca em largura em G que produza T como árvore de largura.

Busca em largura

Exercício: Elaborar uma adaptação da busca em largura, para aplicá-la a digrafos.

Busca em largura

Exercício: Mostre que o algoritmo a seguir funciona como uma busca em profundidade se a estrutura de dados D for uma pilha, e como uma busca em largura se a estrutura de dados D for uma fila.

```
desmarcar todos os vértices
escolher uma raiz  $v$ ;  $\text{pai}(v) \leftarrow \text{null}$ ; marcar  $v$ ; inserir  $v$  em  $D$ 
enquanto  $D \neq \emptyset$  faça
    seja  $v$  o primeiro elemento de  $D$ 
    se existe  $w$  em  $N(v)$  que esteja desmarcado
        então visitar aresta  $vw$ 
             $\text{pai}(w) \leftarrow v$ 
            marcar  $w$ 
            inserir  $w$  em  $D$ 
    senão remover  $v$  de  $D$ 
fim-enquanto
```